

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a33f7d5e1ea734a4c.jsonl`  
- SHA-256: `8355c4ecd0964c5906dc4e372697d18d166a54a525862c572a0556d1649cc375`  
- Source modified: `2026-06-10T06:11:32+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_2a64bc88-2a8`  
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`
```

Transcript

1. user (2026-06-10T06:09:28.019Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

ROLE: adversarial verifier. An auditor ("collector") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.

FINDING: {"title":"validate-delivery passes the QA gate on a CVAT export that parses to ZERO rallies","severity":"high","area":"vendor QA gate","file":"C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py:604-612","evidence":"The --csv path guards emptiness (curate.py:619-621 'no usable rally rows' -> sys.exit(1)) but the --input CVAT path does not. Verified empirically: a CVAT XML with no boxes (or with a misnamed rally_number attribute ? boxes lacking a matched rally_number are silently skipped at rally_cvat.py:212-214) converts to 0 records / 0 issues, so build_summary gives n_blockers=0 and validate-delivery prints 'GATE: PASS' and exits 0. convert-cvat likewise prints 'No blockers. Next: import-local...' for an empty result (curate.py:580-583).","why_it_matters":"The worst delivery failure mode (vendor exported the wrong job / renamed the rally_number attribute / empty annotations) sails through the automated gate that INGESTION_PIPELINE.md positions as the bulk-loop/CI blocker. An expected-rally-count or nonzero-records assertion is missing."}

Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:09:36.475Z)

540

541 Absorbs the CVAT bounding-box delivery format (rally fields carried as box

```

542         attributes) on the ingestion side: each rally's time is recomputed from
543         its frame range at the video's true fps, and content defects (overlaps,
544         zero-duration, off-vocabulary endings, coarse sampling) are written to an
545         issues report rather than silently "fixed". The emitted CSV imports via
546         `import-local`; any blocker issue must be resolved first, since the
547         importer's validator rejects overlapping / zero-duration rallies.
548         """
549         from src.parsers.cvat.rally_cvat import (
550             convert, write_canonical_csv, render_issue_report,
551         )
552         try:
553             result = convert(args.input, fps=args.fps,
554                             video_path=args.video_path, step=args.step)
555         except (FileNotFoundError, ValueError) as e:
556             print(f"Error: {e}")
557             sys.exit(1)
558
559         write_canonical_csv(result.records, args.out)
560
561         # Observability report next to the canonical CSV. Never raises.
562         from src.reporting import emit_delivery_report
563         src_name = os.path.splitext(os.path.basename(args.input))[0]
564         dur = (result.meta.get("stop_frame") / result.fps) if
565         (result.meta.get("stop_frame") and result.fps) else None
566         emit_delivery_report(
567             out_dir=os.path.dirname(os.path.abspath(args.out)) or ".", source=src_name,
568             records=result.records, issues=result.issues, fps=result.fps,
569             video_path=args.video_path,
570             video_duration_s=dur, source_format="cvat", step=result.step,
571         )
572         report = render_issue_report(result, os.path.basename(args.input))
573         if args.issues:
574             with open(args.issues, "w", encoding="utf-8") as f:
575                 f.write(report)
576         print(report)
577         print(f"Canonical CSV written: {args.out} ({len(result.records)} rallies)")
578         if result.n_blockers:
579             print(f"\n {result.n_blockers} BLOCKER issue(s) ? import-local will reject
580             "
581             f"this file until they are resolved (by the vendor or manual
582             review).")
583         else:
584             print("\n No blockers. Next:")
585             print(f"    python -m src.cli.curate import-local --sport <sport> "
586                   f"--video-path <video> --annotations-path {args.out} --fps
587                   {result.fps:g}")
588
589     def validate_delivery(self, args):
590         """Run the full validation suite on a rally delivery and emit a report.
591
592         Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
593         CSV (--csv). Writes, into --out-dir:
594             <id>_report.md      human report (scoreboard + issues + rallies to review)
595             <id>_report.json   machine summary (for CI / a bulk-ingest loop)
596             <id>_review.csv    the annotatable manual-verification sheet
597         Exits non-zero if any blocker issue exists, so it can gate a bulk pipeline.
598         """
599         import json
600         from src.parsers.cvat.rally_cvat import (
601             convert, load_canonical_csv, detect_issues,
602             build_summary, render_validation_md, write_review_csv,
603         )
604         vid = args.video_id or "delivery"
605         fps = None

```

```

602         step = None
603         stop_frame = None
604         if args.input:
605             try:
606                 result = convert(args.input, fps=args.fps, video_path=args.video_path)
607             except (FileNotFoundError, ValueError) as e:
608                 print(f"Error: {e}")
609                 sys.exit(1)
610             records, issues, fps = result.records, result.issues, result.fps
611             step = result.step
612             stop_frame = result.meta.get("stop_frame")
613         elif args.csv:
614             try:
615                 records = load_canonical_csv(args.csv)
616             except FileNotFoundError as e:
617                 print(f"Error: {e}")
618                 sys.exit(1)
619             if not records:
620                 print(f"Error: no usable rally rows in {args.csv}")
621                 sys.exit(1)
622             issues = detect_issues(records)
623         else:
624             print("Error: provide --input <cvat.xml> or --csv <canonical.csv>")
625             sys.exit(1)
626
627         os.makedirs(args.out_dir, exist_ok=True)
628         summary = build_summary(records, issues, source=vid, fps=fps)
629         md = render_validation_md(records, issues, source=vid)
630         json_path = os.path.join(args.out_dir, f"{vid}_report.json")
631         md_path = os.path.join(args.out_dir, f"{vid}_report.md")
632         review_path = os.path.join(args.out_dir, f"{vid}_review.csv")
633         with open(json_path, "w", encoding="utf-8") as f:
634             json.dump(summary, f, indent=2)
635         with open(md_path, "w", encoding="utf-8") as f:
636             f.write(md)
637         write_review_csv(records, issues, review_path)
638
639         # Observability report (envelope JSON + technical md + customer summary),
640         # colocated with the existing artifacts. Never raises.
641         from src.reporting import emit_delivery_report
642         duration_s = (stop_frame / fps) if (stop_frame and fps) else None
643         report_paths = emit_delivery_report(
644             out_dir=args.out_dir, source=vid, records=records, issues=issues, fps=fps,
645             video_path=args.video_path, video_duration_s=duration_s,
646             source_format=("cvat" if args.input else "csv"), step=step,
647         )
648
649         print(md)
650         print(f"\nwrote:\n {md_path}\n {json_path}\n {review_path}")
651         if report_paths:
652             print(f" {report_paths.get('technical_md')}\n
653 {report_paths.get('customer_md')}\n {report_paths.get('json')}")
654             if summary["n_blockers"]:
655                 print(f"\n GATE: BLOCKED ? {summary['n_blockers']} blocker(s) must be
656 resolved.")
657                 sys.exit(2)
658             print("\n GATE: PASS ? review the flagged rallies, then `import-local`.")
659
660     def curate(self):
661         """Interactive loop to review and curate staging video annotations."""

```

3. user (2026-06-10T06:09:37.143Z)

Found 28 files

C:\Users\avidu\Projects\sports-data-collector\tests\test_normalize_annotations.py

```

C:\Users\avidu\Projects\sports-data-collector\src\cli\normalize_annotations.py
C:\Users\avidu\Projects\sports-data-collector\docs\MANUAL_INGESTION_POSITIONING.md
C:\Users\avidu\Projects\sports-data-collector\data\validation\pilot_badminton_01_report.json
C:\Users\avidu\Projects\sports-data-collector\data\validation\pilot_badminton_01_review.csv
C:\Users\avidu\Projects\sports-data-collector\src\reporting\adapter.py
C:\Users\avidu\Projects\sports-data-collector\tests\test_reporting.py
C:\Users\avidu\Projects\sports-data-collector\data\roora_badminton_canonical.csv
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py
C:\Users\avidu\Projects\sports-data-collector\docs\INGESTION_PIPELINE.md
C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat\rally_cvat.py
C:\Users\avidu\Projects\sports-data-collector\tests\test_cvat_parser.py
C:\Users\avidu\Projects\sports-data-collector\tests\test_db.py
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\pickleball.py
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tabletennis.py
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\badminton.py
C:\Users\avidu\Projects\sports-data-collector\src\core\db.py
C:\Users\avidu\Projects\sports-data-collector\docs\annotation\ANNOTATION_GUIDE.md
C:\Users\avidu\Projects\sports-data-collector\docs\annotation\ROORA_BRIEF.md
C:\Users\avidu\Projects\sports-data-collector\tests\test_parser.py
C:\Users\avidu\Projects\sports-data-collector\tests\test_cli.py
C:\Users\avidu\Projects\sports-data-collector\src\parsers\badminton\coachai.py
C:\Users\avidu\Projects\sports-data-collector\src\core\validator.py
C:\Users\avidu\Projects\sports-data-collector\docs\DECISIONS.md
C:\Users\avidu\Projects\sports-data-collector\tests\test_validator.py
C:\Users\avidu\Projects\sports-data-collector\tests\test_local_import.py
C:\Users\avidu\Projects\sports-data-collector\tests\test_schemas.py
C:\Users\avidu\Projects\sports-data-collector\data\mock_local_badminton_anno.json

```

4. user (2026-06-10T06:09:42.430Z)

```

170
171     boxes: List[Dict[str, Any]] = []
172
173     # Image mode: <image id=frame> ... <box>...</box>
174     for img in root.findall("image"):
175         try:
176             frame = int(img.get("id"))
177         except (TypeError, ValueError):
178             continue
179         for box in img.findall("box"):
180             boxes.append({"frame": frame, "attrs": _attrs(box)})
181
182     # Track mode: <track> ... <box frame=N outside=0>...</box>
183     for track in root.findall("track"):
184         for box in track.findall("box"):
185             if box.get("outside") == "1":
186                 continue # a track's terminating keyframe carries no real label
187             try:
188                 frame = int(box.get("frame"))
189             except (TypeError, ValueError):
190                 continue
191             boxes.append({"frame": frame, "attrs": _attrs(box)})
192
193     return meta, boxes
194
195
196 def _infer_step(frames: List[int]) -> int:
197     """Most common positive gap between consecutive distinct sampled frames."""
198     uniq = sorted(set(frames))
199     gaps = [b - a for a, b in zip(uniq, uniq[1:])] if b - a > 0
200     if not gaps:
201         return 1
202     return Counter(gaps).most_common(1)[0][0]
203
204

```

```

205 def build_records(boxes: List[Dict[str, Any]], fps: float) -> List[RallyRecord]:
206     """Group boxes by rally_number and derive each rally's frame range -> seconds."""
207     if fps is None or fps <= 0:
208         raise ValueError("fps must be a positive number to convert frames to seconds")
209
210     by_rally: Dict[int, List[Dict[str, Any]]] = {}
211     for b in boxes:
212         rn = safe_int(_attr_lookup(b["attrs"], "rally_number"), None)
213         if rn is None:
214             continue
215         by_rally.setdefault(rn, []).append(b)
216
217     records: List[RallyRecord] = []
218     for rn in sorted(by_rally):
219         items = by_rally[rn]
220         frames = sorted(b["frame"] for b in items)
221         attrs = items[0]["attrs"]
222         fmin, fmax = frames[0], frames[-1]
223         records.append(RallyRecord(
224             rally_number=rn,
225             start_time=fmin / fps,
226             end_time=fmax / fps,
227             shots_count=safe_int(_attr_lookup(attrs, "shots_count"), None),
228             ending_reason=_attr_lookup(attrs, "ending_reason"),
229             last_shot_outcome=_attr_lookup(attrs, "last_shot_outcome"),
230             score_before=_attr_lookup(attrs, "score_before"),
231             score_after=_attr_lookup(attrs, "score_after"),
232             notes=_attr_lookup(attrs, "notes"),
233             frame_min=fmin,
234             frame_max=fmax,
235             n_boxes=len(frames),
236             vendor_start_time=attrs.get("start_time"),
237             vendor_end_time=attrs.get("end_time"),
238         ))
239     return records
240
241
242 def detect_issues(records: List[RallyRecord],
243                 stop_frame: Optional[int] = None,
244                 allowed_reasons: Optional[set] = None,
245                 allowed_outcomes: Optional[set] = None,
246                 max_gap_s: float = 15.0) -> List[Issue]:
247     """Find the defects that block or degrade a clean ground-truth import.
248
249     Note: we deliberately do NOT flag the sampling density (step/fps). A boundary
250     slack of ~0.5s is acceptable for this pipeline because the downstream
251     highlight compiler pads every clip on output (badminton-highlight-indexer,
252     backend/stitcher/compiler.py: begin_padding=0.5s, end_padding=1.0s), which
253     absorbs the sampling uncertainty when highlights are trimmed.
254     """
255     allowed = allowed_reasons if allowed_reasons is not None else
256     DEFAULT_ALLOWED_REASONS
257     allowed_out = allowed_outcomes if allowed_outcomes is not None else
258     DEFAULT_ALLOWED_OUTCOMES
259     issues: List[Issue] = []
260
261     ordered = sorted(records, key=lambda r: r.rally_number)
262     for i, r in enumerate(ordered):
263         if r.end_time <= r.start_time:
264             issues.append(Issue(
265                 "blocker", "zero_duration",
266                 f"Rally {r.rally_number}: end <= start "
267                 f"({frames {r.frame_min}-{r.frame_max}}); single-sample or stray box.",
268                 r.rally_number))

```

```

268         if r.ending_reason is not None and r.ending_reason not in allowed:
269             issues.append(Issue(
270                 "medium", "ending_reason_vocab",
271                 f"Rally {r.rally_number}: ending_reason '{r.ending_reason}' "
272                 f"not in the agreed set {sorted(allowed)}.",
273                 r.rally_number))
274
275         if r.last_shot_outcome is not None and r.last_shot_outcome not in allowed_out:
276             issues.append(Issue(
277                 "low", "outcome_vocab",
278                 f"Rally {r.rally_number}: last_shot_outcome '{r.last_shot_outcome}' "
279                 f"not in {sorted(allowed_out)}.",
280                 r.rally_number))
281
282         # Inflated span: a long rally with implausibly few shots usually means a
283         # stray/duplicated box stretched the frame range past the real ending.
284         if r.shots_count and r.duration >= 5.0:
285             sps = r.shots_count / r.duration
286             if sps < 0.3:
287                 issues.append(Issue(
288                     "medium", "shot_density",
289                     f"Rally {r.rally_number}: {r.shots_count} shots over "

```

5. assistant (2026-06-10T06:09:54.346Z)

Now let me check the `convert` function for any empty-result guard.

6. user (2026-06-10T06:09:56.120Z)

```

290             f"{r.duration:.1f}s ({sps:.2f}/s) is implausibly low ? the span "
291             f"may be inflated by a stray box.",
292             r.rally_number))
293
294         if stop_frame is not None and r.frame_max > stop_frame:
295             issues.append(Issue(
296                 "high", "frame_out_of_range",
297                 f"Rally {r.rally_number}: frame {r.frame_max} exceeds the video's "
298                 f"last frame {stop_frame}.",
299                 r.rally_number))
300
301         if i > 0:
302             prev = ordered[i - 1]
303             if r.start_time < prev.end_time:
304                 issues.append(Issue(
305                     "blocker", "overlap",
306                     f"Rally {prev.rally_number} (ends {prev.end_time:.2f}s) overlaps "
307                     f"Rally {r.rally_number} (starts {r.start_time:.2f}s) by "
308                     f"{prev.end_time - r.start_time:.2f}s ? rallies must be disjoint.",
309                     r.rally_number))
310             elif r.start_time - prev.end_time > max_gap_s:
311                 # A long unlabeled stretch between two rallies is a cheap, label-only
312                 # second opinion for the worst error (a missed rally). It won't catch a
313                 # rally MERGED into its neighbour's span (shot_density/inflated catches
314                 # that), but it does catch an omitted rally that left a visible hole.
315                 gap = r.start_time - prev.end_time
316                 issues.append(Issue(
317                     "medium", "possible_missed_rally",
318                     f"{gap:.1f}s of unlabeled time between Rally {prev.rally_number} "
319                     f"(ends {prev.end_time:.1f}s) and Rally {r.rally_number} "
320                     f"(starts {r.start_time:.1f}s) ? check for an unlabeled rally in the
321                     gap.",
322                     r.rally_number))
323
324         return issues

```

```

325 def convert(xml_path: str,
326             fps: Optional[float] = None,
327             video_path: Optional[str] = None,
328             step: Optional[int] = None,
329             allowed_reasons: Optional[set] = None) -> ConversionResult:
330     """Full pipeline: parse CVAT XML -> records (+ issues) at the true fps.
331
332     ``fps`` resolution order: explicit ``fps`` arg > ffprobe of ``video_path`` >
333     the CVAT meta (none) -> error. We never fall back to a guessed 30 fps, since
334     that silent guess is exactly the bug this adapter exists to undo.
335     """
336     meta, boxes = parse_cvat_boxes(xml_path)
337
338     if fps is None and video_path:
339         _, _, probed = probe_resolution_fps(video_path)
340         fps = probed
341     if fps is None or fps <= 0:
342         raise ValueError(
343             "Could not determine fps. Pass fps=... or a readable video_path so "
344             "frames can be converted to seconds (seconds = frame / fps).")
345
346     if step is None:
347         step = meta.get("step") or _infer_step([b["frame"] for b in boxes])
348
349     records = build_records(boxes, fps)
350     issues = detect_issues(records,
351                           stop_frame=meta.get("stop_frame"),
352                           allowed_reasons=allowed_reasons)
353     return ConversionResult(records=records, issues=issues, fps=fps, step=step,
354                             meta=meta)
355
356 def write_canonical_csv(records: List[RallyRecord], out_path: str) -> None:
357     """Write records as the canonical per-rally CSV that ``import-local`` reads."""
358     import csv
359     os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
360     with open(out_path, "w", newline="", encoding="utf-8") as f:
361         w = csv.writer(f)
362         w.writerow(CANONICAL_COLUMNS)
363         for r in sorted(records, key=lambda x: x.rally_number):
364             w.writerow([
365                 r.rally_number,
366                 _mmss(r.start_time), _mmss(r.end_time),
367                 f"{r.start_time:.3f}", f"{r.end_time:.3f}",
368                 r.shots_count if r.shots_count is not None else "",
369                 r.ending_reason or "",
370                 r.last_shot_outcome or "",
371                 r.score_before or "", r.score_after or "", r.notes or "",
372             ])
373
374 def render_issue_report(result: ConversionResult, source: str) -> str:
375     """A human-readable issue report (for review / sending back to the vendor)."""
376     lines = [
377         f"CVAT rally conversion report ? {source}",
378         f"  fps used           : {result.fps:.3f}",
379         f"  sampling step      : {result.step} frames (~{result.step / result.fps:.2f}s)",
380         f"  rallies parsed     : {len(result.records)}",
381         f"  issues              : {len(result.issues)} ({result.n_blockers} blocker)",
382         "",
383     ]
384
385     if not result.issues:
386         lines.append("No issues detected.")
387     else:
388         for sev in ("blocker", "high", "medium", "low"):

```

```

389         group = [i for i in result.issues if i.severity == sev]
390         if not group:
391             continue
392         lines.append(f"[{sev.upper()}]")
393         for i in group:
394             lines.append(f"    - ({i.code}) {i.message}")
395         lines.append("")
396     return "\n".join(lines)
397
398
399     # ----- #
400     # Validation reporting ? used by `curate validate-delivery`. #
401     # These turn a (records, issues) pair into the artifacts a human reviews: #
402     # a machine summary (JSON), a human report (md), and an annotatable sheet. #
403     # ----- #
404
405     def _parse_secs(value: Optional[str]) -> Optional[float]:
406         """Decimal seconds or mm:ss / hh:mm:ss -> float; None if blank/unparseable."""
407         s = (value or "").strip()
408         if not s:
409             return None
410         if ":" in s:
411             try:
412                 total = 0.0
413                 for part in s.split(":"):
414                     total = total * 60 + float(part)
415                 return total
416             except ValueError:
417                 return None
418         try:
419             return float(s)

```

7. user (2026-06-10T06:09:57.739Z)

```

325: def convert(xml_path: str,
427:     fallback; rows with no usable start/end are skipped. (frame_* are -1 here, as a

```

8. user (2026-06-10T06:10:12.582Z)

```

463: def build_summary(records: List[RallyRecord], issues: List[Issue],
464-     source: str = "", fps: Optional[float] = None) -> Dict[str, Any]:
465-     """Machine-readable validation summary (JSON / CI)."""
466-     by_sev = {s: [i for i in issues if i.severity == s]
467-               for s in ("blocker", "high", "medium", "low")}
468-     flagged = sorted([i.rally_number for i in issues if i.rally_number is not None])
469-     return {
470-         "source": source,
471-         "fps": fps,
472-         "n_rallies": len(records),
473-         "n_issues": len(issues),
474-         "counts_by_severity": {s: len(v) for s, v in by_sev.items()},
475-         "n_blockers": len(by_sev["blocker"]),
476-         "passed": len(by_sev["blocker"]) == 0,
477-         "flagged_rallies": flagged,
478-         "issues": [{"severity": i.severity, "code": i.code,
479-                    "rally_number": i.rally_number, "message": i.message} for i in issues],
480-     }
481-
482-
483: def render_validation_md(records: List[RallyRecord], issues: List[Issue], source: str) ->
str:
484-     """Human validation report: scoreboard + issues by severity + rallies to review."""
485-     n = len(records)
486-     by_sev = {s: [i for i in issues if i.severity == s]
487-               for s in ("blocker", "high", "medium", "low")}

```

```

488-     flagged = sorted({i.rally_number for i in issues if i.rally_number is not None})
489-     clean = n - len(flagged)
490-     lines = [
491-         f"# Delivery validation ? {source}",
492-         "",
493-         f"- rallies: **{n}**",
494-         f"- issues: **{len(issues)}** (blocker {len(by_sev['blocker'])}, "
495-         f"high {len(by_sev['high'])}, medium {len(by_sev['medium'])}, low
496-         f"{len(by_sev['low'])})",
497-         f"- structurally-clean rallies: **{clean}/{n}**"
498-         + (f" ({100 * clean // n}%" if n else ""),
499-         f"- gate: **{'PASS' if not by_sev['blocker'] else 'BLOCKED'}**",
500-         "",
501-     ]
502-     if issues:
503-         for sev in ("blocker", "high", "medium", "low"):
504-             grp = by_sev[sev]

```

9. user (2026-06-10T06:10:13.739Z)

```

32:# 2. Validate the delivery ? report + annotatable review sheet (the QA gate)
33:python -m src.cli.curate validate-delivery --input job.xml \
36:python -m src.cli.curate validate-delivery --csv canonical.csv --video-id pilot_badminton_01
78:### Blocker vs. review ? what's the gate?
79:`validate-delivery` exits **non-zero if any blocker exists** (so it can gate a bulk
80:loop / CI). Non-blocker issues don't stop the gate but populate the **review sheet**
97:(timestamp overlay on) and fill the flagged rows. This is the human gate that turns
108:2. `convert-cvat` (if CVAT) ? `validate-delivery` ? read the report, fill the review
132:Every `validate-delivery`, `convert-cvat`, and `import-local` run now also writes a

```

10. user (2026-06-10T06:10:24.707Z)

```

120
121     @property
122     def n_blockers(self) -> int:
123         return sum(1 for i in self.issues if i.severity == "blocker")
124
125
126     def _mmss(seconds: float) -> str:
127         """Format seconds as ``m:ss.mmm`` (matches the brief's mm:ss columns)."""
128         m = int(seconds // 60)
129         s = seconds - m * 60
130         return f"{m}:{s:06.3f}"
131
132
133     def _attr_lookup(attrs: Dict[str, str], canonical: str) -> Optional[str]:
134         for alias in _ATTR_ALIASES[canonical]:
135             if alias in attrs and attrs[alias] != "":
136                 return attrs[alias]
137         return None
138
139
140     def parse_cvat_boxes(xml_path: str) -> Tuple[Dict[str, Any], List[Dict[str, Any]]]:
141         """Parse a CVAT XML export into (meta, boxes).
142
143         ``boxes`` is a flat list of ``{"frame": int, "attrs": {name: value}}`` ? one
144         entry per visible box, from either image-mode or track-mode exports.
145         ``meta`` carries ``step`` (frame_filter) and ``stop_frame`` when present.
146         """
147         if not os.path.exists(xml_path):
148             raise FileNotFoundError(f"CVAT export not found: {xml_path}")
149
150         root = ET.parse(xml_path).getroot()
151         meta: Dict[str, Any] = {}
152

```

```

153     job = root.find("./job")
154     if job is not None:
155         stop = job.findtext("stop_frame")
156         if stop is not None and stop.strip().isdigit():
157             meta["stop_frame"] = int(stop.strip())
158         ff = job.findtext("frame_filter") or ""
159         if "step=" in ff:
160             try:
161                 meta["step"] = int(ff.split("step=")[1].split()[0])
162             except (ValueError, IndexError):
163                 pass
164
165     def _attrs(el: ET.Element) -> Dict[str, str]:
166         out: Dict[str, str] = {}
167         for a in el.findall("attribute"):
168             out[(a.get("name") or "").strip().lower()] = (a.text or "").strip()
169         return out
170
171     boxes: List[Dict[str, Any]] = []

```

11. user (2026-06-10T06:10:26.728Z)

C:\Users\avidu\Projects\sports-data-collector\docs\MANUAL_INGESTION_POSITIONING.md:43:Verified end-to-end on the real file ? 40 rallies ? export ? 40-interval indexer CSV.

12. user (2026-06-10T06:10:28.134Z)

```

124: def test_zero_duration_is_a_blocker():
127:     assert any(i.code == "zero_duration" and i.severity == "blocker" for i in issues)
207: def test_load_canonical_csv_keeps_rally_starting_at_zero(tmp_path):
218:     assert len(recs) == 3 # the zero-start row 1 is not lost

```

13. assistant (2026-06-10T06:10:49.563Z)

Let me check the `\_ATTR\_ALIASES` for rally_number (to assess the "misnamed attribute" sub-claim) and confirm there is no zero-record check anywhere else in the gate path.

14. user (2026-06-10T06:10:51.604Z)

```

65: _ATTR_ALIASES = {
66:     "rally_number": ("rally_number",),
67:     "shots_count": ("shots_count",),
68:     "ending_reason": ("ending_reason",),
69:     "last_shot_outcome": ("last_shot_outcome", "outcome", "last_shot_outcome (in/net/out)"),
70:     "score_before": ("score_before", "score_before (a-b)"),
71:     "score_after": ("score_after", "score_after (a-b)"),
72:     "notes": ("notes", "note"),
73: }
74:
75:
76: "rally_number", "start_mmss", "end_mmss", "start_time", "end_time",
77: "shots_count", "ending_reason", "last_shot_outcome", "score_before", "score_after",
78: "notes",
79: ]
80:
81: @dataclass
82: class RallyRecord:
83:     --
212:     rn = safe_int(_attr_lookup(b["attrs"], "rally_number"), None)
213:     if rn is None:
214:         continue
215:     by_rally.setdefault(rn, []).append(b)
216:
217: records: List[RallyRecord] = []
218: for rn in sorted(by_rally):

```

```

--
439:         rn = safe_int(row.get("rally_number"), None)
440:         if rn is None:
441:             continue
442:         start = _parse_secs(row.get("start_time"))
443:         if start is None:
444:             start = _parse_secs(row.get("start_mmss"))
445:         end = _parse_secs(row.get("end_time"))
--
479:         "rally_number": i.rally_number, "message": i.message} for i in issues],
480:     }
481-
482-
483-def render_validation_md(records: List[RallyRecord], issues: List[Issue], source: str) ->
str:
484:     """Human validation report: scoreboard + issues by severity + rallies to review."""
485:     n = len(records)
--
524:     cols = ["rally_number", "start_mmss", "end_mmss", "dur_s", "shots_count",
525:             "ending_reason", "last_shot_outcome", "auto_flags",
526:             "VERIFIED(Y/N)", "true_start_mmss", "true_end_mmss",
527:             "true_ending_reason", "true_last_shot_outcome", "reviewer_comment"]
528:     with open(out_path, "w", newline="", encoding="utf-8") as f:
529:         w = csv.writer(f)
530:         w.writerow(cols)

```

15. user (2026-06-10T06:10:52.742Z)

```

92:         is_valid, errors = self.validator.validate_badminton_rallies(annotations)
128:         count = self.db.insert_badminton_rallies(annotations)
619:         if not records:
701:         rallies = self.db.get_badminton_rallies(video.video_id)

```

16. user (2026-06-10T06:11:26.254Z)

Structured output provided successfully

17. assistant (2026-06-10T06:11:32.275Z)

Verification complete.